西安电子科技大学

微机原理与系统设计 课程实验报告

实验名称 实验一 汇编语言编程实验

计算机科学与技术学院 1903011 班

姓名 许珈铭 学号 19050500032

同作者

实验日期 年 月 日

实验地点 E-II 522 实验批次 5

成 绩

指导教师评语：

指导教师：

年 月 日

实验报告内容基本要求及参考格式

一、实验目的

二、实验所用仪器（或实验环境）

三、实验基本原理及步骤（或方案设计及理论计算）

四、实验数据记录（或仿真及软件设计）

五、实验结果分析及回答问题（或测试环境及测试结果）

实验一　汇编语言编程实验

一、实验目的

1. 掌握汇编语言的编程方法；
2. 掌握 DOS 功能调用的使用方法；
3. 掌握汇编语言程序的调试运行过程。

二、实验内容

1. 在屏幕上显示自己的学号姓名信息。

2. 将指定数据区的字符串数据以 ASCII 码形式显示在屏幕上，并通过 DOS 功能调用完成必要提示信息的显示。

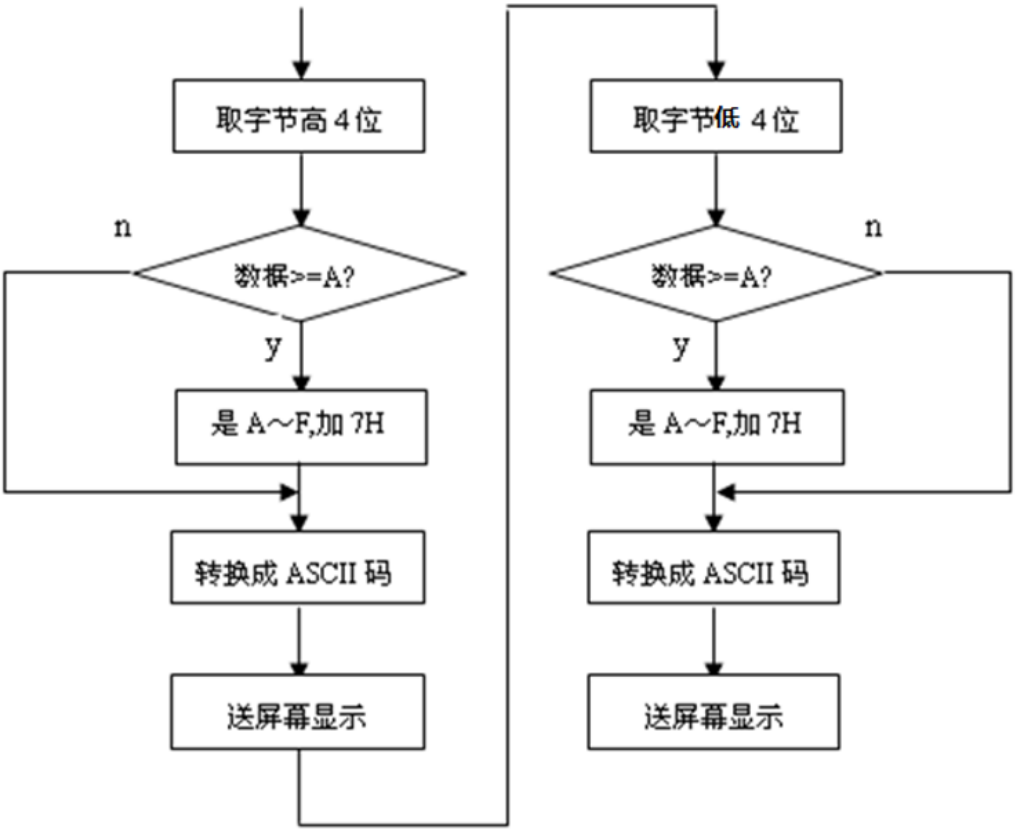
3. 循环从键盘读入字符并回显在屏幕上，然后显示出对应字符的 ASCII 码，直到输入”Q”或“q”时结束。

4. 自主设计输入显示信息，完成编程与调试，演示实验结果。

三、实验步骤

1. 运行星研软件，根据实验内容，参考如下程序流程图编写程序。

字符转换为ASCII码流程图



代码如下：

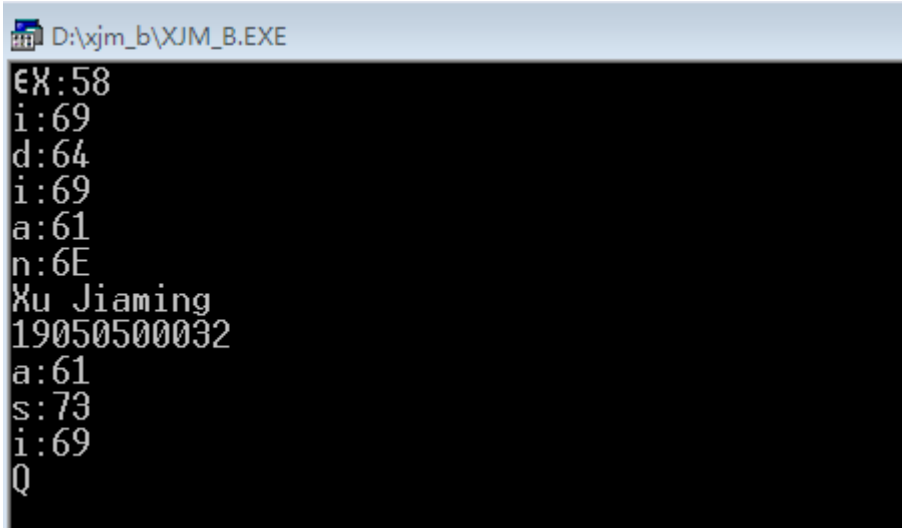
```
mystack segment stack
  db 100 dup(?)
mystack ends

data segment
  s db 'helloworld$'
  len equ $-s
  crlf db 0dh, 0ah, '$'
  nam db 'Xujiaming$'
  idn db '19050500032$'
  ma db 'Xidian'
data ends

code segment
assume     cs:code,     ds:data,
ss:mystack
start:
  mov ax, data
  mov ds, ax
  mov cx, len
  sub cx, 2
  mov bl, cl
  mov ah, 2
  int 21h
  mov bx, 0
  L1:
      mov dl, [s+bx]
      mov ah, 2
      int 21h
      mov dl, ':'
      mov ah, 2
      int 21h
      mov al, [s+bx]
      mov ah, 0
      mov dh, 16
      div dh
      push bx
          mov bh, 0
          mov bl, al
          mov dl, [ma+bx]
      pop bx

      push ax
          mov ah, 2
          int 21h
      pop ax

      push bx
          mov bl, ah
          mov bh, 0
          mov dl, [ma+bx]
          mov ah, 2
          int 21h
      pop bx

      lea dx, crlf
      mov ah, 9
      int 21h

      inc bx
      cmp bx, cx
      jbe L1
  lea dx, nam
  mov ah, 9
```

```
int 21h

lea dx, crlf
mov ah, 9
int 21h

lea dx, idn
mov ah, 9
int 21h

lea dx, crlf
mov ah, 9
int 21h

L2:
    mov ax, 0
    mov ah, 1
    int 21h

    cmp al, 'q'
    jz over

    cmp al, 'Q'
    jz over

    push ax
        mov dl, ':'
        mov ah, 2
        int 21h
    pop ax

    mov ah, 0
    mov dh, 16
    div dh

    mov bh, 0
    mov bl, al
    mov dl, [ma+bx]

    push ax
        mov ah, 2
        int 21h
    pop ax

    mov bl, ah
    mov bh, 0
    mov dl, [ma+bx]
    mov ah, 2
    int 21h

    lea dx, crlf
    mov ah, 9
    int 21h

    jmp L2
over:

    mov ax, 4c00h
    int 21h
code ends
end start
```

2. 选择“项目”菜单中的 “编译连接”对实验程序进行编译连接。

3.使用“运行”菜单中的“进行调试”命令进入调试状态，执行单步调试或全速运行，观察调试过程中寄存器窗、观察光和变量窗等信息窗内各寄存器及数据区的内容，在信息窗的DOS窗口可显示和输入相应信息和数据。

4.运行对应的.exe文件可以展示效果如下：



首先展示的对应的存储区的内容‘Xidian’对应的各个字母的ASCII码值，然后输出我的名字和学号，最后输入想要查询ASCII码值的字符，自动输出对应的ASCII码值，直到输入‘Q’或‘q’结束。

四、实验心得

通过本次实验，我掌握了基础的星研软件编程和调试的基本用法，为接下来的编程和硬件操作打下了基础。同时，我借此机会温习了汇编语言编程的相关语言用法，加深了对汇编语言的熟悉程度。

西安电子科技大学

微机原理与系统设计 课程实验报告

实验名称 实验二 数码转换实验

计算机科学与技术学院 1903011 班

姓名 许珈铭 学号 19050500032

同作者

实验日期 年 月 日

成 绩

实验地点 E-II 522 实验批次 5

指导教师评语：

指导教师：

年 月

日

实验报告内容基本要求及参考格式

一、实验目的

二、实验所用仪器（或实验环境）

三、实验基本原理及步骤（或方案设计及理论计算）

四、实验数据记录（或仿真及软件设计）

五、实验结果分析及回答问题（或测试环境及测试结果）

实验二　数码转换实验

一、实验目的

1. 掌握不同进制数及编码相互转换的程序设计方法。
2. 掌握运算类指令编程及调试方法。
3. 掌握循环程序的设计方法。

二、实验内容

1. 重复从键盘输入不超过 5 位的十进制数，按回车键结束输入；
2. 将该十进制数转换成二进制数；结果以 2 进制数的形式显示在屏幕上；
3. 如果输入非数字字符，则报告出错信息，重新输入；
4. 直到输入“Q”或‘q’时程序运行结束。
5. 键盘输入一字符串，以空格结束，统计其中数字字符的个数，在屏幕显示

三、实验原理

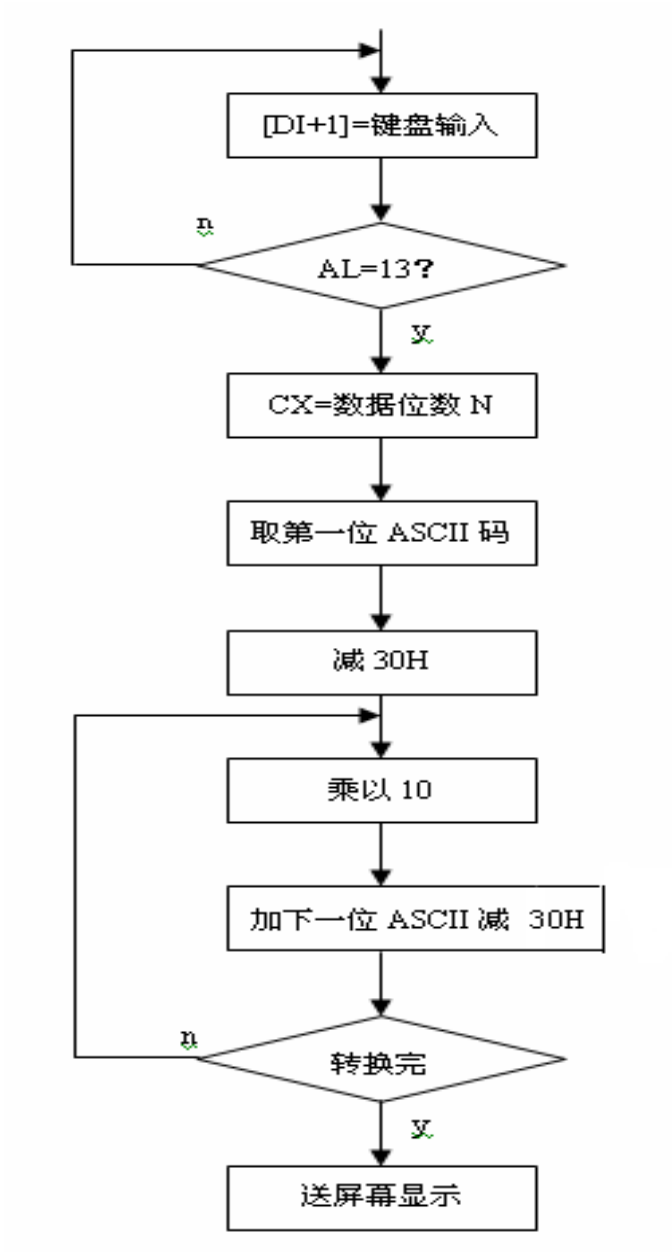
十进制数可以表示为：$D_n*10^n+D_{n-1}*10^{n-1}+\cdots+D_0*10^0=\Sigma\ D_i*10^i$，其中 D_i 代表十进制数 1、2、3、…、9、0。

上式可以转换为：$\Sigma\ D_i*10^i=(((D_n*10+D_{n-1})*10+D_{n-2})*10+\cdots+D_1)*10+D_0$

由上式可归纳出十进制数转换为二进制数的方法：从十进制数的最高位 D_n 开始做乘 10 加次位的操作，依此类推，则可求出二进制数结果。

四、实验步骤

根据如下的十进制 ASCII 码转换为二进制数流程图编写程序。



代码如下：

```
_STACK SEGMENT PARA STACK'_STACK'
   DB 128 DUP(0)
_STACK ENDS
DATA SEGMENT
hello DB 'Input number or instruction!!Q OR q: EXIT,s:Search,enter to end input!',0AH,0DH,'$'
wrong DB 0AH,0DH,'Wrong Input(Remember only numbers!)',0AH,0DH,'$'
wrong2 DB 0AH,0DH,'TOOMORE CHARS, ONLY HANDLE FIRST 5 CHARS!',0AH,0DH,'$'
```

```
endofhex DB 0AH,0DH,'IN HEX OVER! NOW IN BINARY!',0AH,0DH,'$'
finish DB 0AH,0DH,'FINISHED!!!',0AH,0DH,'$'
hello2 DB 0AH,0DH,'Search number inyour string. Space to end input',0AH,0DH,'Input string:',0AH,0DH,'$'
finish2 DB 0AH,0DH,'FINISHED!!!',0AH,0DH,'THERE ARE ','$'
finish3 DB ' numbers',0AH,0DH,'$'
got DB 5 DUP(0)
DATA ENDS

CODE SEGMENT
assume cs:CODE,ds:DATA,ss:_STACK

start:
  beginofread:
      mov ax,DATA
      mov ds,ax

      mov dx,offset hello
      mov ah,09H
      int 21H

      mov bx,0H
      mov di,offset got
      mov cx,0H

  readchar:
      mov ah,01H
      int 21H
      cmp bx,0H
      jne notfirst
      cmp al,'Q'
      je exitclose
      cmp al,'q'
```

```
    je exitclose
    cmp al,'s'
    je counterofnumberclose
notfirst:
    mov bx,01H
    call legalcheck
    cmp bx,02H
    je beginofread
    cmp bx,04H
    je endofinput
    jmp loadinmemory
loadinmemory:
    mov [di],al
    inc cx
    inc di
    jmp readchar
endofinput:
    mov dx,0H
    mov di,offset got
beginofhandle:
    mov bx,0H
    mov bl,[di]
    sub bx,30H
    add dx,bx
    cmp cx,1H
    je endofhandle
    call mulAHdxtodx
    dec cx
    inc di
    jmp beginofhandle
endofhandle:
    call hexoutput
    call binaryoutput
```

```
    jmp beginofread

counterofnumberclose:
    jmp counterofnumber

exitclose:
    jmp exit

hexoutput:
    mov bx,dx
    mov ax,bx
    and ah,0F0H
    mov cl,04H
    shr ah,cl
    call hexoutputah
    mov ax,bx
    and ah,0FH
    call hexoutputah
    mov ax,bx
    and al,0F0H
    mov cl,04H
    shr al,cl
    call hexoutputal
    mov ax,bx
    and al,0FH
    call hexoutputal
    mov dl,'H'
    mov ah,02H
    int 21H
    mov dx,offset endofhex
    mov ah,09H
    int 21H
    mov dx,bx
```

```
    ret


binaryoutput:
    mov bx,dx
    mov dx,0H
    mov cx,10H;
beginofoutputloop:
    shl bx,1
    jnc out0
    mov dl,'1'
    jmp outputdl
out0:
    mov dl,'0'
    outputdl:
    mov ah,02H
    int 21H
    dec cx
    cmp cx,0H
    jne beginofoutputloop
    mov dx,offset finish
    mov ah,09H
    int 21H
    ret

legalcheck:
    cmp al,0DH
    je endlegalnextline
    cmp al,30H
    jb endlegalfalse
    cmp al,39H
    ja endlegalfalse
endlegaltrue:
```

```
    mov bx,03H
    ret
endlegalnextline:
    mov bx,04H
    ret
endlegalfalse:
    mov dx,offset wrong
    mov ah,09H
    int 21H
    mov bx,02H
    ret


mulAHdxtodx:
    mov bx,0H
    mov ax,0H
loopofmul:
    add ax,dx
    inc bx
    cmp bx,0AH
    jb loopofmul
    mov dx,ax
    ret

hexoutputah:
    cmp ah,10
    jb hexoutputahbelow
    add ah,07H
hexoutputahbelow:
    add ah,30H
    mov dl,ah
    mov ah,02H
    int 21H
```

```
    ret

hexoutputal:
    cmp al,10
    jb hexoutputalbelow
    add al,07H
hexoutputalbelow:
    add al,30H
    mov dl,al
    mov ah,02H
    int 21H
    ret

counterofnumber:
    mov dx,offset hello2
    mov ah,09H
    int 21H
    mov cx,0H
beginofcount:
    mov ah,01H
    int 21H
    cmp al,20H
    je endofcount
    cmp al,30H
    jb notnum
    cmp al,39H
    ja notnum
isnum:
    inc cx
    jmp  beginofcount
notnum:
    jmp beginofcount
```

```
    endofcount:
        add cx,30H
        mov dx,offset finish2
        mov ah,09H
        int 21H
        mov dx,0H
        mov dl,cl
        mov ah,02H
        int 21H
        mov dx,offset finish3
        mov ah,09H
        int 21H
        jmp beginofread

    exit:
        mov ah,4CH
        int 21H

CODE ENDS
END START
```

2. 选择“项目”菜单中的 “编译连接”对实验程序进行编译连接。

3.使用“运行”菜单中的“进行调试”命令进入调试状态，执行单步调试或全速运行，观察调试过程中寄存器窗、观察光和变量窗等信息窗内各寄存器及数据区的内容，在信息窗的DOS窗口可显示和输入相应信息和数据。

4.运行对应的.exe 文件可以展示效果如下：

```
D:\>D:\test
Input number or instruction!!Q OR q: EXIT,s:Search,enter to end input!
12321
3021H
IN HEX OVER! NOW IN BINARY!
0011000000100001
FINISHED!!!
Input number or instruction!!Q OR q: EXIT,s:Search,enter to end input!
s
Search number inyour string. Space to end input
Input string:
asdjio12378qweji
FINISHED!!!
THERE ARE 5 numbers
```

首先展示的是提示语“Input number or instruction!! Q or q: EXIT, s:Search, enter to end input”表示输入数字或者指令，指令主要有两个：Q 或 q 表示退出，s 表示 搜索功能即输入一串字符串搜索其中有多少数字。

输入数字后，会输出对应的十六进制数和二级制数，不断输入直到输入 Q 或 q 或 s 进入相应的功能。

输入 s 后输入对应一串字符串，然后输出这串字符串中有多少数字。

五、实验心得

通过本次实验，我掌握了基础的星研软件编程和调试的基本用法，为接下来的编程和硬件操作打下了基础。同时又加深了我对汇编语言编程的逻辑用法，为了完成实验，我学习了不同进制数及编码相互转换的程序设计方法、运算类指令编程及调试方法和循环程序的设计方法。

西安电子科技大学

微机原理与系统设计 课程实验报告

实验名称 实验三 基本 I0 口扩展实验

计算机科学与技术学院 1903011 班

姓名 许珈铭 学号 19050500032

同作者

实验日期 年 月 日

实验地点 E-II 522 实验批次 5

成 绩

指导教师评语：

指导教师：

年 月 日

实验报告内容基本要求及参考格式

一、实验目的

二、实验所用仪器（或实验环境）

三、实验基本原理及步骤（或方案设计及理论计算）

四、实验数据记录（或仿真及软件设计）

五、实验结果分析及回答问题（或测试环境及测试结果）

实验三　基本 IO 口扩展实验

一、实验目的

1. 了解 TTL 芯片扩展简单 I/O 口的方法；
2. 掌握数据输入输出程序编制的方法。

二、实验内容

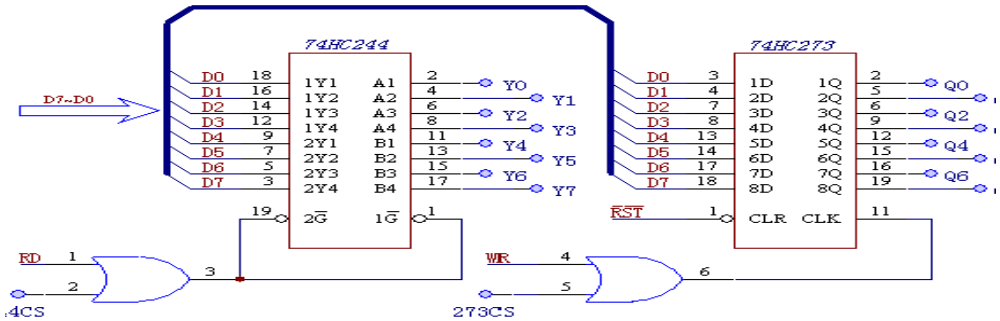
本实验要求用 74LS244 作为输入口，读取开关状态，并将此状态通过 74LS273 连到发光二极管显示。具体实验内容如下：

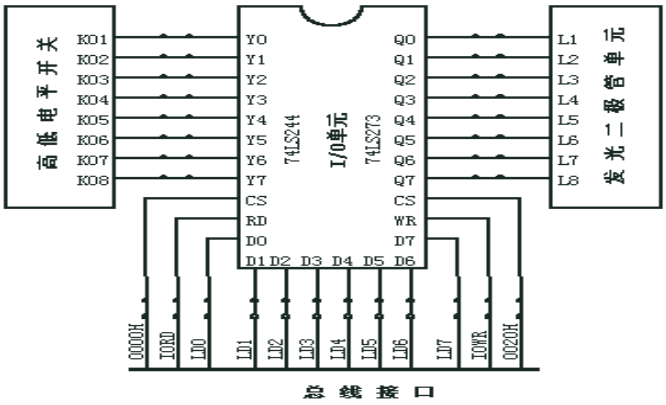
1. 当开关 Yi 为低电平时对应的发光二极管亮，Yi 为高电平时对应的发光二极管灭。
2. 当开关 Yi 全为高电平时，发光二极管 Qi 从左至右轮流点亮。
3. 当开关 Yi 全为低电平时，发光二极管 Qi 从右至左轮流点亮。
4. 自主设计控制及显示模式，完成编程调试，演示实验结果。

三、实验原理

1. 74LS244 是一种三态输出的 8 总线缓冲驱动器，无锁存功能，当 G 为低电平，Ai 信号传送到 Yi，当为高电平时，Yi 处于禁止高阻状态；

2. 74LS273 是一种带清除功能的 8D 触发器，　1D～8D 为数据输入端，1Q～8Q 为数据输出端，正脉冲触发，低电平清除，常用作 8 位地址锁存器。





四、实验步骤

1. 按照实验连线图连接：

- B4 模块 244 的 CS244——总线模块 A3 的 CS1，输入数据信号 IN0—IN7（JP52）——开关（JP27）K1—K8；
- B4 模块 273 的 CS273——总线模块 A3 的 CS2，输出数据信号 O0—O7（JP51）——发光二极管（JP18）L1—L8；
- B4 模块的 WR、RD 分别连到总线模块 A3 的 IOW、IOR；
- B4 模块的数据信号 D0—D7（JP57）连到总线模块的数据信号 D0—D7（JP42）。

2. 运行星研软件，根据实验内容，参考如下程序流程图编写程序。

代码如下：

```
IO244       equ 230h        ;244(16 位)
片选
IO273       equ 230h        ;273(16 位)
片选

_stack segment  stack
          dw 100 DUP(?)
_stack      ends

_data segment   word public 'DATA'
_data       ends

code segment
main proc
            assume
cs:code,ds:_data,ss:_stack
            mov     ax,_data
            mov     ds,ax
            mov     bx,0fefeh
    start:  mov     dx,IO244
            in      al,dx
            mov     ah,al
            cmp     al,0C0h
            jnz     Next5
            call    Add_3;
            jmp     Next2
    Next5:  cmp     al,0fh
            jnz     Next4
            call    RandL
            jmp     Next2
    Next4:  cmp     al,0f0h
            jnz     Next3
            call    LandR
            jmp     Next2
    Next3:      cmp             al,0ffh
    ;全 1,
            jnz     Next0
            call    RORight
            jmp     Next2
    Next0:      cmp             al,0
    ;全 0
            jnz     Next1
            call    ROLeft
            jmp     Next2
```

```
    Next1:  call   Light
    Next2:  call   Delay
            jmp    start
main endp
ADD_3 proc    near
        shl    al,4
        shr    al,4
        shr    ah,4
        add    al,ah
        mov    ah,0
        mov    dx,IO273
        out    dx,ax
        ret
ADD_3 endp
RandL proc    near
        ror    bl,1
        mov    al,bl
        rol    bh,1
        mov    ah,bh
        mov    dx,IO273
        out    dx,ax
        ret
RandL endp

LandR proc    near
        rol    bl,1
        mov    al,bl
        ror    bh,1
        mov    ah,bh
        mov    dx,IO273
        out    dx,ax
        ret
LandR endp
ROLeft proc near
            rol    bl,1
            mov    al,bl
            mov    dx,IO273
            out    dx,ax
            ret
ROLeft endp

RORight proc near
            ror    bl,1
            mov    al,bl
            mov    dx,IO273
            out    dx,ax
            ret
RORight endp

Light proc near
                ;mov    bh,0feh
            xor    bh,1
            mov    al,bh
            mov    dx,IO273
            out    dx,ax
            ret
Light endp

Delay           proc           near
        ;延时
            xor    cx,cx
    delay1: loop   delay1
            ret
Delay endp
code ends
END main
```

3. 选择“项目”菜单中的 “编译连接”对实验程序进行编译连接。

4. 拨动开关，观察发光二极管的变化。具体演示已经通过验收。

演示结果是：

五、实验心得

通过本次实验，我掌握了基础的星研软件编程和调试的基本用法，为接下来的编程和硬件操作打下了基础。同时，我借此机会温习了汇编语言编程的相关语言用法，加深了对汇编语言的熟悉程度。

西安电子科技大学

微机原理与系统设计 课程实验报告

实验名称 实验四 可编程并行接口实验

计算机科学与技术学院 1903011 班

姓名 许珈铭 学号 19050500032

同作者

实验日期 年 月 日

实验地点 E-II 522 实验批次 5

成 绩

指导教师评语：

指导教师：

年 月 日

实验报告内容基本要求及参考格式

一、实验目的

二、实验所用仪器（或实验环境）

三、实验基本原理及步骤（或方案设计及理论计算）

四、实验数据记录（或仿真及软件设计）

五、实验结果分析及回答问题（或测试环境及测试结果）

实验四　可编程并行接口实验

一、实验目的

1. 了解可编程并行接口 8255 的内部结构；

2. 编程使用 8255 芯片实现简单的输入输出控制。

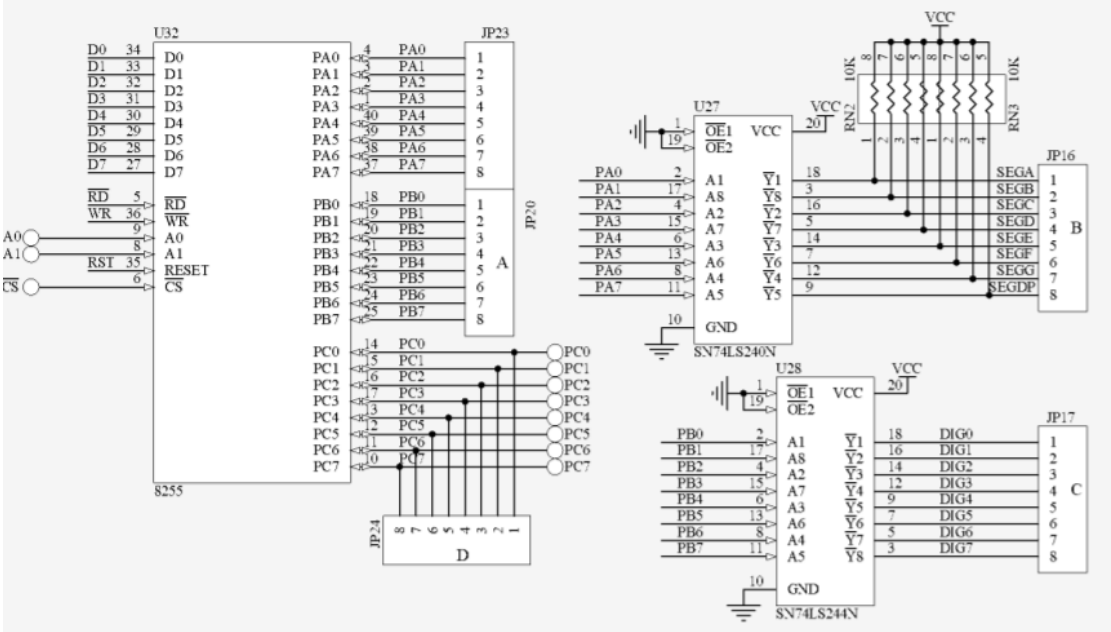
二、实验内容

1. 利用 8255 的 A 口读取拨动开关状态，利用 8255 的 B 口将对应位状态用发光二极管显示。拨动开关为低电平时，对应的发光二极管亮；拨动开关为高电平时，对应的发光二极管灭。

2. 利用 8255 的 A 口读取拨动开关状态。拨动开关为全低电平时，发光二极管从左至右流水灯点亮；拨动开关为全高电平时，发光二极管从右至左流水灯点亮。

3. 自主发挥设计控制及显示模式。

三、实验原理



四、实验步骤

1. 硬件连接：连接片选；连接 8255 端口选择信号；连接相关数据信号。

2. 硬件连接：连接片选；连接 8255 端口选择信号；连接相关数据信号。运行星研软件，根据实验内容，参考如下程序流程图编写程序。

代码如下：

```
COM_ADD     equ 0273H
PA_ADD      equ 0270h
PB_ADD      equ 0271h
PC_ADD      equ 0272h
_stack segment  stack
          dw 100 DUP(?)
_stack      ends

_data segment   word public 'DATA'
_data       ends

code segment
main proc
            assume
cs:code,ds:_data,ss:_stack
            mov    ax,_data
            mov    ds,ax
            mov    dx,COM_ADD
            mov    al,10010000B
            out    dx,al
    start:  mov    dx, PA_ADD
            in     al,dx
            mov    ah,al
            cmp    al,0C0h
            jnz    Next5
            call   Add_3;
            jmp    Next2
    Next5:  cmp    al,0fh
            jnz    Next4
            call   RandL
            jmp    Next2
    Next4:  cmp    al,0f0h
            jnz    Next3
            call   LandR
            jmp    Next2
    Next3:      cmp           al,0ffh
    ;全1,
            jnz    Next0
            call   RORight
            jmp    Next2
    Next0:      cmp             al,0
    ;全0
            jnz    Next1
            call   ROLeft
            jmp    Next2
    Next1:  call   Light
    Next2:  call   Delay
            jmp    start
main endp
ADD_3 proc    near
        shl    al,4
        shr    al,4
        shr    ah,4
        add    al,ah
        mov    ah,0
        mov    dx, PB_ADD
        out    dx,ax
        ret
ADD_3 endp
RandL proc    near
        ror    bl,1
        mov    al,bl
        rol    bh,1
        mov    ah,bh
```

```
        mov    dx,IO273
        out    dx,ax
        ret
RandL endp

LandR proc    near
        rol    bl,1
        mov    al,bl
        ror    bh,1
        mov    ah,bh
        mov    dx, PB_ADD
        out    dx,ax
        ret
LandR endp
ROLeft proc near
            rol    bl,1
            mov    al,bl
            mov    dx, PB_ADD
            out    dx,ax
            ret
ROLeft endp

RORight proc near
            ror    bl,1
            mov    al,bl
            mov    dx, PB_ADD
            out    dx,ax
            ret
RORight endp

Light proc near
                ;mov    bh,0feh
            xor    bh,1
            mov    al,bh
            mov    dx,PB_ADD
            out    dx,ax
            ret
Light endp

Delay           proc           near
        ;延时
            xor    cx,cx
    delay1: loop   delay1
            ret
Delay endp
code ends
END main
```

3. 选择“项目”菜单中的 “编译连接”对实验程序进行编译连接。

四、代码解释以及实现原理

根据实验要求,可以得出,因为实验四和实验三的实现内容相似,只是将 273 和 244 转用 8255 芯片来进行实现，其他实验现象和效果并没有什么辩护啊，所以只需要对实验三中 IO 接口进行改变，改成对应的 8255 中的 PA、PB、PC 接口即可。

1. 现对接口地址进行定义，使用的是 EQU 语句。
2. 然后进入主程序 main,先对 8255 进行设置,设置参数为 10010000，从参数可以看出，A 为入口，B、C 为输出。
3. 开始循环以下内容：从 PA_ADD（A 口）读取数据，对数据进行判断，对于全 0，全 1，11000000，以及其他的情况进行判定。
4. 全 0：有对应的效果函数执行，是产生前 8 个发光二极管从左至右流水灯点亮，后 8 个流水灯从右至左流水灯点亮。
5. 全 1：有对应的效果函数执行，是产生前 8 个发光二极管从右至左流水灯点亮，后 8 个流水灯从左至右流水灯点亮。
6. 11000000:执行加法函数，1100+0000，流水灯展示 00001100 的效果
7. 其他情况：执行展示函数，前 8 个发光二极管按照输入情况进行展示，0 为灭，1 为亮。

五、实验心得

通过本次实验，我复习了 8255 芯片的简单工作原理，用代码实现了其初始化、工作方式及应用的功能，同时基于实验三的 IO 输入输出的控制，我编程掌握了通过 8255 来实现 IO 输入输出的简单实现。